

Technical Requirement Document

Here is the Technical Requirement Document (TRD) based on the provided Functional Requirement Document (FRD):

TR-DLT-001: Load Customer and Order Data into Delta Tables

Technical Requirement ID: TR-DLT-001

Related Functional Requirement(s): FR-DLT-001

Objective: Implement a Delta Live Table to load customer and order data from CSV files into Delta tables.

Target Cluster Configuration:

- Cluster Name: Delta Live Table Cluster
- Databricks Runtime Version: 10.4.x-scala2.12
- Node Type: Standard_DS3_v2
- Driver Node: Standard_DS3_v2
- Worker Nodes: 2-4 Standard_DS3_v2
- Autoscaling: Enabled
- Auto Termination: 30 minutes
- Libraries Installed: Delta Lake, PySpark

Source Data Details:

- Dataset Name: Customer Data
- Location: ``/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata``
- Format: CSV
- Description: Customer data in CSV format
- Dataset Name: Order Data
- Location: ``/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata``
- Format: CSV
- Description: Order data in CSV format

Target Data Details:

- Output Name: Customer Delta Table
- Location: ``gen_ai_poc_databrickscoe.sdlc_wizard.customer``
- Format: Delta
- Description: Customer data in Delta format

- Output Name: Order Delta Table
- Location: `gen\_ai\_poc\_databrickscoe.sdmc\_wizard.order`
- Format: Delta
- Description: Order data in Delta format

Job Flow / Pipeline Stages:

- Read customer CSV data into a Delta table
- Read order CSV data into a Delta table
- Add a new column "TotalAmount" to the "order" Delta table

Data Transformations / Business Logic:

- Step: Read customer CSV data
- Description: Read customer CSV data into a DataFrame
- Transformation Logic: Use PySpark to read CSV data
- Step: Read order CSV data
- Description: Read order CSV data into a DataFrame
- Transformation Logic: Use PySpark to read CSV data
- Step: Add "TotalAmount" column
- Description: Add a new column "TotalAmount" to the "order" Delta table
- Transformation Logic: Use PySpark to add a new column with the product of "PricePerUnit" and "Qty" columns

Error Handling and Logging:

- Log errors to a logging table
- Handle null and duplicate records

TR-DLT-002: Clean and Process Customer and Order Data

Technical Requirement ID: TR-DLT-002

Related Functional Requirement(s): FR-DLT-002

Objective: Implement data cleaning and processing for customer and order Delta tables.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details: Same as TR-DLT-001

Target Data Details: Same as TR-DLT-001

Job Flow / Pipeline Stages:

- Remove null records from customer and order Delta tables
- Remove duplicate records from customer and order Delta tables

Data Transformations / Business Logic:

- Step: Remove null records
- Description: Remove null records from customer and order Delta tables
- Transformation Logic: Use PySpark to filter out null records
- Step: Remove duplicate records
- Description: Remove duplicate records from customer and order Delta tables
- Transformation Logic: Use PySpark to drop duplicate records

Error Handling and Logging:

- Log errors to a logging table

TR-DLT-003: Create ordersummary Table and Load Data

Technical Requirement ID: TR-DLT-003

Related Functional Requirement(s): FR-DLT-003

Objective: Implement a Delta Live Table to create an "ordersummary" table and load data into it.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details:

- Dataset Name: Customer Delta Table
- Location: `gen\_ai\_poc\_databrickscoe.sdlc\_wizard.customer`
- Format: Delta
- Description: Customer data in Delta format
- Dataset Name: Order Delta Table
- Location: `gen\_ai\_poc\_databrickscoe.sdlc\_wizard.order`
- Format: Delta
- Description: Order data in Delta format

Target Data Details:

- Output Name: ordersummary Table
- Location: `gen\_ai\_poc\_databrickscoe.sdlc\_wizard.ordersummary`
- Format: Delta
- Description: ordersummary data in Delta format

Job Flow / Pipeline Stages:

- Create the "ordersummary" table if it does not exist
- Join customer and order Delta tables on the "CustId" field
- Load the joined data into the "ordersummary" table as an SCD Type 2 table

Data Transformations / Business Logic:

- Step: Join customer and order data
- Description: Join customer and order Delta tables on the "CustId" field
- Transformation Logic: Use PySpark to join DataFrames
- Step: Load data into ordersummary table
- Description: Load the joined data into the "ordersummary" table as an SCD Type 2 table
- Transformation Logic: Use PySpark to load data into a Delta table with SCD Type 2 logic

Error Handling and Logging:

- Log errors to a logging table

TR-DLT-004: Update ordersummary Table on Customer Changes

Technical Requirement ID: TR-DLT-004

Related Functional Requirement(s): FR-DLT-004

Objective: Implement a Delta Live Table to update the "ordersummary" table on customer changes.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details:

- Dataset Name: Customer Delta Table
- Location: `gen\_ai\_poc\_databrickscoe.sdlc\_wizard.customer`
- Format: Delta
- Description: Customer data in Delta format

Target Data Details:

- Output Name: ordersummary Table
- Location: `gen\_ai\_poc\_databrickscoe.sdlc\_wizard.ordersummary`
- Format: Delta
- Description: ordersummary data in Delta format

Job Flow / Pipeline Stages:

- Identify changes to customer data
- Update the "ordersummary" table by making old records inactive and new records active
- Update the "StartDate" and "EndDate" fields accordingly to maintain history

Data Transformations / Business Logic:

- Step: Identify customer changes
- Description: Identify changes to customer data
- Transformation Logic: Use PySpark to compare DataFrames
- Step: Update ordersummary table

- Description: Update the "ordersummary" table by making old records inactive and new records active
- Transformation Logic: Use PySpark to update a Delta table with SCD Type 2 logic

Error Handling and Logging:

- Log errors to a logging table

TR-DLT-005: Create customeragggregatespend Table and Load Data

Technical Requirement ID: TR-DLT-005

Related Functional Requirement(s): FR-DLT-005

Objective: Implement a Delta Live Table to create a "customeragggregatespend" table and load aggregated data into it.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details:

- Dataset Name: ordersummary Table
- Location: `gen\_ai\_poc\_databrickscoe.sdmc\_wizard.ordersummary`
- Format: Delta
- Description: ordersummary data in Delta format

Target Data Details:

- Output Name: customeragggregatespend Table
- Location: `gen\_ai\_poc\_databrickscoe.sdmc\_wizard.customeragggregatespend`
- Format: Delta
- Description: customeragggregatespend data in Delta format

Job Flow / Pipeline Stages:

- Create the "customeragggregatespend" table if it does not exist
- Aggregate the "TotalAmount" column from the "ordersummary" table by "Name" and "Date" columns
- Load the aggregated data into the "customeragggregatespend" table

Data Transformations / Business Logic:

- Step: Aggregate data
- Description: Aggregate the "TotalAmount" column from the "ordersummary" table by "Name" and "Date" columns
- Transformation Logic: Use PySpark to aggregate a DataFrame
- Step: Load data into customeragggregatespend table
- Description: Load the aggregated data into the "customeragggregatespend" table
- Transformation Logic: Use PySpark to load data into a Delta table

Error Handling and Logging:

- Log errors to a logging table

TR-DLT-006: Implement Delta Live Tables

Technical Requirement ID: TR-DLT-006

Related Functional Requirement(s): FR-DLT-006

Objective: Implement Delta Live Tables to ensure data is processed and loaded correctly.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details: Same as previous TRs

Target Data Details: Same as previous TRs

Job Flow / Pipeline Stages:

- Implement Delta Live Tables for all previous TRs

Data Transformations / Business Logic:

- Same as previous TRs

Error Handling and Logging:

- Log errors to a logging table